

Tensor and Pipeline Parallelism

Splitting one model across workers · video 2:29

This lesson answers one question: how do you train a model that doesn't fit on one worker? Split the model itself. **Tensor parallelism** splits each weight matrix: two workers each compute half of an MLP and an all-reduce sums their partial outputs (matching the full MLP to 1.2×10^{-6}). **Pipeline parallelism** splits the layers: one worker sends its activations to the next (an exact match), and micro-batches shrink the idle “bubble”. Real runs combine both with data parallelism.

How to use this guide: master one concept before moving to the next.

1. Watch the whole video once, without stopping.
2. Then take the concepts **in order**. For each one: read the short explanation, look at the picture from the video (▶ is the time in the video), and answer every question in its *Check* without looking anything up.
3. Check your answers in the **Answers** section at the end (page 5).
4. **Pass mark: every question in the check correct.** If you missed one, rewatch that concept's video segment, reread the explanation, and redo only the questions you missed. Tick the concept off below, then move on.

It is strongly suggested that you **master each concept before starting the next one**. Each concept builds on the previous ones.

Concepts in this lesson

CONCEPT	VIDEO SEGMENT	QUESTIONS	MASTERED
1. Tensor parallelism	▶ 0:08-1:16	2	☐
2. Pipeline parallelism and the bubble	▶ 1:16-1:55	2	☐
3. Combining them	▶ 1:55-2:18	2	☐

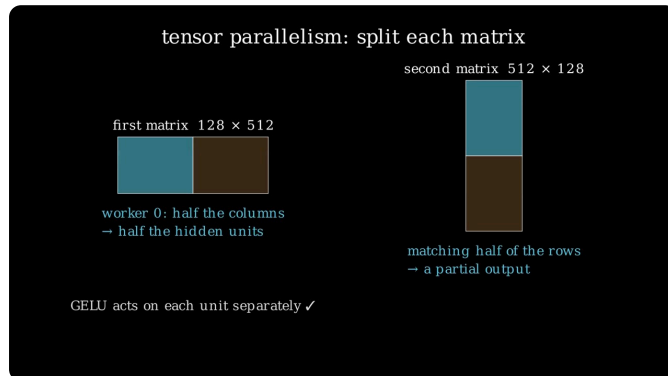
Before you start: Deep Dive episode 25 (data parallelism) and How LLMs Work episode 8 (the MLP). Code: `code/d26_tensor_pipeline` (PyTorch only; starts 2 local processes; seconds on a CPU).

CONCEPT 1 OF 3

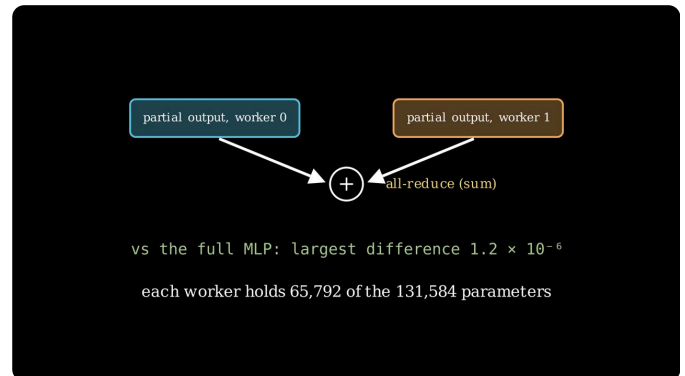
Tensor parallelism

▶ Video 0:08–1:16

Data parallelism needs the whole model on every worker; a 7-billion-parameter model with Adam needs 112 GB. So split the model. **Tensor parallelism** splits each weight matrix. For an MLP $128 \rightarrow 512 \rightarrow 128$, worker 0 keeps the first half of the **columns** of the first matrix, so it computes half the hidden units; GELU acts on each unit separately, so that is fine. It keeps the matching half of the **rows** of the second matrix and gets a partial output. Worker 1 does the same with the other halves.



▶ 0:41 Worker 0 keeps half the columns of the first matrix (half the hidden units) and the matching half of the rows of the second.



▶ 0:58 Summing the two partial outputs (all-reduce) gives the full MLP's output, to 1.2×10^{-6} .

Summing the two partial outputs with an all-reduce gives exactly the full MLP's output: in a real run with two processes the largest difference is 1.2×10^{-6} (float rounding), and each worker holds 65,792 of the 131,584 parameters. The price is an all-reduce of the layer's output in every layer (256 KiB here; attention is split by heads the same way), so tensor parallelism usually stays inside one machine, where links between GPUs are fastest.

KEY IDEA

Split the first matrix by columns and the second by rows: each worker produces a partial sum, and one all-reduce completes it.

Check 1: Tensor parallelism

Answer all 2 questions before looking at the answers. Pass mark: all correct.

1.1 CALCULATION

With 4 tensor-parallel workers, how many of the 512 hidden units does each compute?

.....

1.2 SHORT ANSWER

Why does splitting the hidden units work with GELU in between?

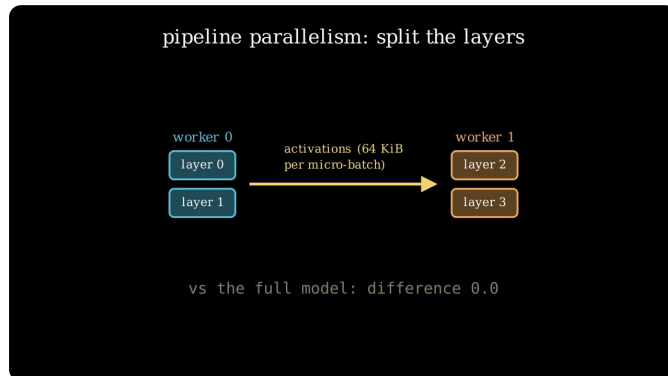
.....

CONCEPT 2 OF 3

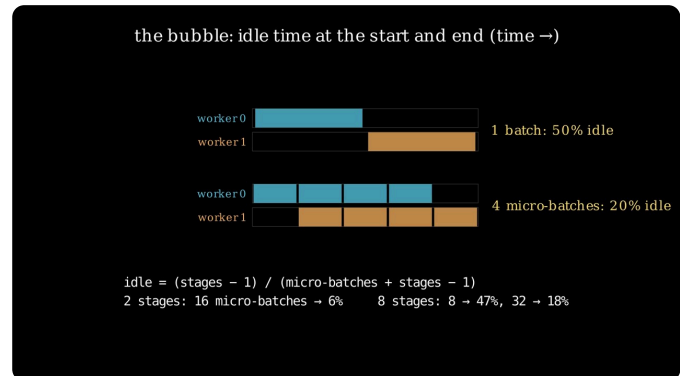
Pipeline parallelism and the bubble

▶ Video 1:16–1:55

Pipeline parallelism splits the layers: worker 0 runs layers 0 and 1, then sends its activations to worker 1, which runs layers 2 and 3. Only one tensor crosses between them (64 KiB per micro-batch here), and the output matches the full model exactly (difference 0.0).



▶ 1:32 Worker 0 runs layers 0–1 and sends its activations (64 KiB per micro-batch) to worker 1 for layers 2–3: an exact match.



▶ 1:53 With 1 batch, each worker idles half the time; with 4 micro-batches, 20%.

But a pipeline has a **bubble**: with one batch, worker 1 waits while worker 0 works, then worker 0 waits: half the time idle. Splitting the batch into micro-batches keeps both busy. The idle fraction is $(\text{stages} - 1) / (\text{micro-batches} + \text{stages} - 1)$: 2 stages with 4 micro-batches, 20%; with 16, 6%; 8 stages need many more (8 micro-batches: 47%; 32: 18%).

KEY IDEA

Pipelines communicate little but idle at the start and end; more micro-batches shrink the bubble.

Check 2: Pipeline parallelism and the bubble

Answer all 2 questions before looking at the answers. Pass mark: all correct.

2.1 CALCULATION

4 stages and 12 micro-batches: what fraction of the time is each stage idle?

2.2 CALCULATION

With 8 stages, how many micro-batches are needed to bring the bubble under 10%?

CONCEPT 3 OF 3

Combining them

▶ Video 1:55-2:18

Real training runs combine all three: **tensor** parallelism inside a machine (heavy, frequent communication on fast links), **pipeline** parallelism across machines (little communication), and **data** parallelism on top, over many copies of the whole arrangement. In code, tensor parallelism is a slice of each matrix and one all-reduce; pipeline parallelism is a send and a receive between stages.

real runs combine all three

tensor parallel: inside a machine

pipeline parallel: across machines

data parallel: many copies of the whole arrangement

▶ 2:06 Tensor parallel inside a machine, pipeline across machines, data parallel on top.

```
1 # tensor parallel, worker k of 2
2 half = slice(k * 256, (k + 1) * 256)
3 partial = F.gelu(x @ w1[:, half] + b1[half]) @ w2[half, :]
4 dist.all_reduce(partial) # sum of halves = full output
5 out = partial + b2
6
7 # pipeline parallel
8 dist.send(h, dst=1) # worker 0 → worker 1
9 dist.recv(h, src=0)
```

▶ 2:16 In code: a slice of each matrix and one all-reduce; a send and a receive.

KEY IDEA

Put the most communication-hungry split on the fastest links.

Check 3: Combining them

Answer all 2 questions before looking at the answers. Pass mark: all correct.

3.1 MULTIPLE CHOICE

Which split communicates in every layer?

- A. Data parallelism
- B. Tensor parallelism
- C. Pipeline parallelism
- D. None

3.2 TRUE OR FALSE

“Pipeline parallelism changes the model's output.”

True ☐ False ☐

ANSWERS

Answers and explanations

Mark each answer right or wrong. If a concept has any wrong answer, rewatch its segment (▶) and redo the questions you missed before moving on.

1. Tensor parallelism · rewatch ▶ 0:08-1:16

1.1 128. $512 / 4$, each worker keeping a quarter of the columns and rows.

1.2 GELU is applied to each hidden unit independently, so each worker can apply it to its own half without needing the other half. A function that mixed hidden units (like a softmax over them) would need communication first.

2. Pipeline parallelism and the bubble · rewatch ▶ 1:16-1:55

2.1 20%. $(4 - 1) / (12 + 4 - 1) = 3 / 15$.

2.2 64. $7 / (m + 7) < 0.1 \rightarrow m > 63$.

3. Combining them · rewatch ▶ 1:55-2:18

3.1 B. Every split layer ends with an all-reduce of its output; pipelines communicate only between stages.

3.2 False. Same layers, same order: the episode's pipeline matches the full model exactly.